Department of Computer Science & Information Technology

University of Balochistan, Quetta

Course Title: Web Engineering

Assignment No: 01

Total Marks: 20

Distribution Date: 20th May 2026

Submission Deadline: 3rd June 2026 (Before 11:59 PM PKT)

Project Objective

Google provides a seamless, real-time user experience for everyday utility tasks, such as its built-in Unit Converter. This assignment tests your mastery of client-side web development—specifically DOM manipulation, event handling, data structures, and responsive design—by requiring you to replicate this core utility tool.

You will design and implement a fully client-side **Universal Unit Converter Web Application** using only semantic **HTML5**, modern **CSS3**, and vanilla **JavaScript (ES6+)**.

Functional Requirements

Your application must closely mimic the workflow and responsiveness of Google's unit conversion interface. It must fulfill the following technical and functional behaviors:

1. Category Selection

- Provide a clean mechanism (such as a sidebar, a grid of icons, or a prominent dropdown menu) to switch between at least **four** distinct conversion categories:
- **Length** (e.g., Millimeters, Centimeters, Meters, Kilometers, Inches, Feet, Yards, Miles)
- **Mass/Weight** (e.g., Milligrams, Grams, Kilograms, Pounds, Ounces)
- **Temperature** (e.g., Celsius, Fahrenheit, Kelvin)
- **Digital Storage** (e.g., Bits, Bytes, Kilobytes, Megabytes, Gigabytes, Terabytes)

2. Dual-Way Dynamic Conversion Interface

- For the selected category, present two symmetrical interaction blocks (Left/Right or Top/Bottom).
- Each block must consist of:
- An **Input Field** (numerical values only).
- A **Dropdown/Select Menu** displaying the available units for that specific category.
- **Bi-directional Binding:** Typing a number into the left input must instantly update the right input based on the chosen units, and vice versa.
- **Instantaneous Response:** Conversion must happen in real-time using event listeners (`input`, `change`). *No "Submit" or "Convert" button is allowed.*

3. Mathematical Accuracy & Formula Display

- Use explicit, accurate conversion factors for all metric, imperial, and digital transformations.
- Account for non-multiplicative conversions (e.g., Temperature offsets like $F = C \times \frac{9}{5} + 32$).
- Dynamically display the mathematical formula or a simple conversion text scale below the fields based on the currently selected units (e.g., *"1 Meter = 3.28 Feet"*).

Technical Constraints & Stack

- **HTML5:** Mark up your page using proper semantic tags (`<header>`, `<main>`, `<section>`, `<nav>`, `<label>`). Ensure form inputs are properly bound to labels for accessibility.
- **CSS3:**
- Design an intuitive, minimalist user interface inspired by modern utility tools.
- You must implement a fully **responsive layout** using CSS Flexbox or CSS Grid so that the converter scales elegantly across desktop monitors, tablets, and mobile smartphones.
- External CSS frameworks (such as Bootstrap, Tailwind, or Bulma) are **strictly prohibited**. All styles must be written in a native, external stylesheet (`style.css`).
- **Vanilla JavaScript:**

- Implement clean, modular script files. Use JavaScript objects or maps to store categories, units, and conversion formulas neatly.
- No external JavaScript libraries or frameworks (such as jQuery, React, or Vue) are permitted.

Grading Rubric (Total: 20 Marks)

Criteria	Description	Marks
Semantic HTML & Structure	Correct use of semantic layout structures, accessible form controls, and clean element separation.	**03**
UI/UX & CSS Responsiveness	Visual layout alignment, typography, clean UI styling, and a seamless responsive design across varying screen sizes.	**05**
Core JavaScript Logic	Dynamic unit menu population based on selected categories, robust bidirectional parsing, and proper multi-unit calculations.	**06**
Real-time Event Execution	Flawless operational performance using live event handling without latency, bugs, or page reloads.	**04**
Code Code Cleanliness & Organization	Readable code architecture, semantic variable naming, code modularity, and concise descriptive commenting.	**02**

Submission Guidelines

1. Organize your codebase into a clean, single project directory structure containing exactly three primary files (or standard subfolders):

```
unit-converter-assignment/
index.html
```

`./css/style.css`
`./js/sript.js`

2. Upload your assignment to github and email the link to your repository on imran.cs.uob@gmail.com. In the subject line mention your Roll Number e.g. 2022-BS(IT)-dd.

   Academic Integrity Notice: Plagiarism, copying code from classmates, or generating unverified third-party templates will result in an immediate score of zero for all involved parties. Write your own clean code.
